Sri Sivasubramaniya Nadar College of Engineering, Chennai
(An autonomous Institution affiliated to Anna University)

Degree & Branch	B.E. Computer Science & Engineering	Semester	V
Subject Code & Name	ICS1512 & Machine Learning Algorithms Laboratory		
Academic year	2025-2026 (Odd)	Batch:2023-2028	**Due date: 30.07.2025**
Name	Ponsubash Raj R	Register No.	**3122237001037**

Experiment 2: Loan Amount Prediction using Linear Regression

1 Aim

To predict the sanctioned loan amount using Linear Regression based on the historical loan data provided.

2 Libraries Used

- `pandas`
- `numpy`
- `matplotlib`
- `seaborn`
- `sklearn`

3 Objective

To apply Linear Regression for estimating loan amounts, perform EDA, and visualize the model performance using appropriate metrics and plots.

4 Mathematical Description

1. Linear Regression

Linear Regression models the relationship between a dependent variable y and independent variables $x_1, x_2, \ldots, x_n$ by fitting a linear equation of the form:

$$\hat{y} = w_0 + w_1 x_1 + w_2 x_2 + \cdots + w_n x_n$$

Where:

- $\hat{y}$ is the predicted output
- w_0 is the intercept (bias term)

- $w_1, w_2, \ldots, w_n$ are the coefficients (weights) for the respective features

The coefficients are chosen to minimize the **Mean Squared Error (MSE)** between actual values y_i and predicted values $\hat{y}_i$:

$$\text{MSE} = \frac{1}{m} \sum_{i=1}^{m} (y_i - \hat{y}_i)^2$$

—

2. Ridge Regression (L2 Regularization)

Ridge Regression modifies the linear regression cost function by adding a penalty on the square of the coefficients:

$$\text{Cost} = \frac{1}{m} \sum_{i=1}^{m} (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^{n} w_j^2$$

Where:

- λ is the regularization parameter that controls the penalty strength
- w_j are the model coefficients (excluding the intercept)

This helps in reducing overfitting and multicollinearity by shrinking the coefficient values.

—

3. Lasso Regression (L1 Regularization)

Lasso Regression adds a penalty on the absolute value of the coefficients:

$$\text{Cost} = \frac{1}{m} \sum_{i=1}^{m} (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^{n} |w_j|$$

Unlike Ridge, Lasso can shrink some coefficients exactly to zero, effectively performing feature selection.

—

4. Mean Squared Error (MSE)

$$\text{MSE} = \frac{1}{m} \sum_{i=1}^{m} (y_i - \hat{y}_i)^2$$

MSE penalizes larger errors more significantly than smaller ones due to squaring. It is commonly used to evaluate regression models.

—

5. Mean Absolute Error (MAE)

$$\text{MAE} = \frac{1}{m} \sum_{i=1}^{m} |y_i - \hat{y}_i|$$

MAE gives the average absolute difference between actual and predicted values. It is less sensitive to outliers than MSE.

—

6. Root Mean Squared Error (RMSE)

$$\text{RMSE} = \sqrt{\frac{1}{m} \sum_{i=1}^{m} (y_i - \hat{y}_i)^2}$$

RMSE is the square root of MSE and has the same unit as the target variable. It is useful when interpreting the error in the original scale.

—

7. Coefficient of Determination (R^2 Score)

$$R^2 = 1 - \frac{\sum_{i=1}^{m} (y_i - \hat{y}_i)^2}{\sum_{i=1}^{m} (y_i - \bar{y})^2}$$

Where:

- $\bar{y}$ is the mean of actual values
- The numerator is the residual sum of squares
- The denominator is the total sum of squares

R^2 represents the proportion of variance in the dependent variable that is predictable from the independent variables.

—

8. Adjusted R^2 Score

$$\text{Adjusted } R^2 = 1 - \left(1 - R^2\right) \cdot \frac{m - 1}{m - n - 1}$$

Where:

- m is the number of observations
- n is the number of predictors (features)

Adjusted R^2 accounts for the number of predictors and helps prevent overestimating the goodness of fit when adding more features.

5 Code with Plot

```
"""# CONFIGURATIONS"""

DATASET_PATH = path
TRAIN_FILE = "train.csv"
TEST_FILE = "test.csv"

# Target variable
TARGET_COL = "Loan Sanction Amount (USD)"

# Columns to drop
UNNECESSARY_COLS = ['Customer ID', 'Name', 'Property ID']

# Categorical filling strategy
CATEGORICAL_FILL_MODE = "mode"

# Numerical fill defaults
NUMERIC_FILL_VALUES = {
    'Credit Score': 'median',
    'Property Age': 'median',
    'Income (USD)': 'median',
    'Loan Amount Request (USD)': 'median',
    'Current Loan Expenses (USD)': 0,
    'No. of Defaults': 0,
    'Dependents': 0
}

# Outlier threshold
OUTLIER_FILTER = {
    'Income (USD)': 1000000
}

from sklearn.linear_model import LinearRegression, Lasso, Ridge
from sklearn.svm import SVR

option = int(input(
    "Choose the regression model:\n"
    "1. Linear Regression\n"
    "2. Support Vector Regressor (SVR)\n"
    "3. Lasso Regression\n"
    "4. Ridge Regression\n"
))

model_param_map = {
    1: {
        "model": LinearRegression(),
```

```
        "params": {
            "copy_X": [True, False],
            "fit_intercept": [True, False],
            "positive": [True, False]
        }
    },
    2: {
        "model": SVR(),
        "params": [
            {"kernel": ["linear", "poly", "rbf", "sigmoid"]}
        ]
    },
    3: {
        "model": Lasso(random_state=42, max_iter=10000),
        "params": {
            "alpha": [0.01, 0.1, 1, 10, 100],
            "fit_intercept": [True, False],
            "positive": [True, False],
            "selection": ["cyclic", "random"]
        }
    },
    4: {
        "model": Ridge(random_state=42, max_iter=10000),
        "params": {
            "alpha": [0.01, 0.1, 1, 10, 100],
            "fit_intercept": [True, False],
            "positive": [True, False]
        }
    }
}

model_config = model_param_map[option]
base_model = model_config["model"]
param_grid = model_config["params"]

"""# IMPORT FUNCTIONS"""

import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import KFold
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.model_selection import train_test_split
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import GridSearchCV
```

```
"""# LOAD DATA"""

df = pd.read_csv(f"{DATASET_PATH}/{TRAIN_FILE}")
print("Shape:", df.shape)

"""# REMOVE UNWANTED COLUMNS"""

if UNNECESSARY_COLS:
    df.drop(columns=[col for col in UNNECESSARY_COLS if col in df.columns], inplace=True)

"""# FILL MISSING VALUES"""

for col, val in NUMERIC_FILL_VALUES.items():
    if col in df.columns:
        if val == 'median':
            df[col] = df[col].fillna(df[col].median())
        elif val == 'mean':
            df[col] = df[col].fillna(df[col].mean())
        else:
            df[col] = df[col].fillna(val)

for col in df.select_dtypes(include='object').columns:
    if CATEGORICAL_FILL_MODE == "mode":
        df[col] = df[col].fillna(df[col].mode()[0])
    else:
        df[col] = df[col].fillna("Unknown")

df = df[~df[TARGET_COL].isna()]
# df = df[df['Loan Sanction Amount (USD)'] > 0]

"""# CHECK AND REMOVE OUTLIERS"""

for col, threshold in OUTLIER_FILTER.items():
    df = df[df[col] < threshold]

non_numerical_cols = df.select_dtypes(include=['object']).columns.tolist()
corr = df.drop(columns=[TARGET_COL] + non_numerical_cols).corr()

# Plot heatmap
plt.figure(figsize=(24, 10))
sns.heatmap(corr, annot=True, cmap="coolwarm", fmt=".2f", cbar=True)
plt.title("Feature Correlation Matrix", fontsize=16)
plt.show()

"""# ENCODE CATEGORICAL DATA"""

label_encoders = {}
for col in df.select_dtypes(include='object').columns:
```

```
    le = LabelEncoder()
    df[col] = le.fit_transform(df[col])
    label_encoders[col] = le

"""# SPLITTING TO FEATURES AND TARGETS"""

X = df.drop(columns=[TARGET_COL])
y = df[TARGET_COL]

"""# Validation"""

# Adjusted R² function
def adjusted_r2(r2, n, k):
    return 1 - (1 - r2) * (n - 1) / (n - k - 1)

# Step 1: Split into Train, Validation, Test
X_temp, X_test, y_temp, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

scaler = StandardScaler()
numerical_cols = X_temp.select_dtypes(include=['int64', 'float64']).columns.tolist()
X_temp[numerical_cols] = scaler.fit_transform(X_temp[numerical_cols])

X_train, X_val, y_train, y_val = train_test_split(X_temp, y_temp, test_size=0.5, random_state=4

"""# CROSS VALIDATION"""

kf = KFold(n_splits=5, shuffle=True, random_state=42)

grid = GridSearchCV(estimator=base_model,
                    param_grid=param_grid,
                    scoring="r2",
                    cv=kf,
                    n_jobs=-1,
                    verbose=3)
grid.fit(X_val, y_val)

print("Best hyperparameters:", grid.best_params_)

best_model = grid.best_estimator_

metrics = {
    "Fold": [], "MAE": [], "MSE": [], "RMSE": [], "R2": [], "AdjR2": []
}

fold = 1
for train_idx, val_idx in kf.split(X_train, y_train):
    X_train_fold, X_val_fold = X_train.iloc[train_idx], X_train.iloc[val_idx]
    y_train_fold, y_val_fold = y_train.iloc[train_idx], y_train.iloc[val_idx]
```

```
    best_model.fit(X_train_fold, y_train_fold)
    y_pred = best_model.predict(X_val_fold)

    r2_test = r2_score(y_val_fold, y_pred)
    metrics["Fold"].append(f"Fold {fold}")
    metrics["MAE"].append(mean_absolute_error(y_val_fold, y_pred))
    metrics["MSE"].append(mean_squared_error(y_val_fold, y_pred))
    metrics["RMSE"].append(np.sqrt(metrics["MSE"][-1]))
    metrics["R2"].append(r2_test)
    metrics["AdjR2"].append(adjusted_r2(r2_test, len(y_val_fold), X_val_fold.shape[1]))

    fold += 1

# Add average
metrics["Fold"].append("Average")
for key in list(metrics.keys())[1:]:
    metrics[key].append(np.mean(metrics[key]))

metrics_df = pd.DataFrame(metrics)
metrics_df

results = pd.DataFrame(grid.cv_results_)

metrics_table = results[
    ["params", "mean_test_score", "std_test_score"]
]

metrics_table

X_test[numerical_cols] = scaler.transform(X_test[numerical_cols])

best_model = grid.best_estimator_

best_model.fit(X_train, y_train)

sets = {
    "Train": (X_train, y_train),
    "Validation": (X_val, y_val),
    "Test": (X_test, y_test)
}

for name, (X, y) in sets.items():
    y_pred = best_model.predict(X)
    r2 = r2_score(y, y_pred)
    adj_r2 = adjusted_r2(r2, len(y), X.shape[1])

    print(f"\n{name} Metrics:")
```

```
print(f"  MAE      : {mean_absolute_error(y, y_pred):.4f}")
print(f"  MSE      : {mean_squared_error(y, y_pred):.4f}")
print(f"  RMSE     : {np.sqrt(mean_squared_error(y, y_pred)):.4f}")
print(f"  R²       : {r2:.4f}")
print(f"  Adj R²   : {adj_r2:.4f}")
```

6 Support Vector Regressor

Kernel	Test R2 Score
linear	0.267226
poly	-0.067687
rbf	-0.068376
sigmoid	-0.054069

7 Included Plots

7.1 Histogram / Distribution Plot of Loan Amount

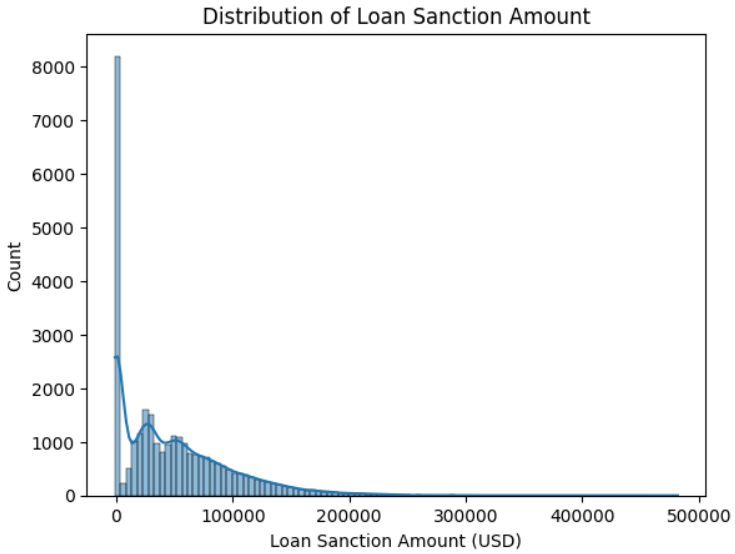

Figure 1: Distribution of Loan Amounts

Interpretation: The histogram shows that the loan amount distribution is right-skewed. Most of the values is found to be having value of $0.

7.2 Scatter Plot of Income vs Loan Amount

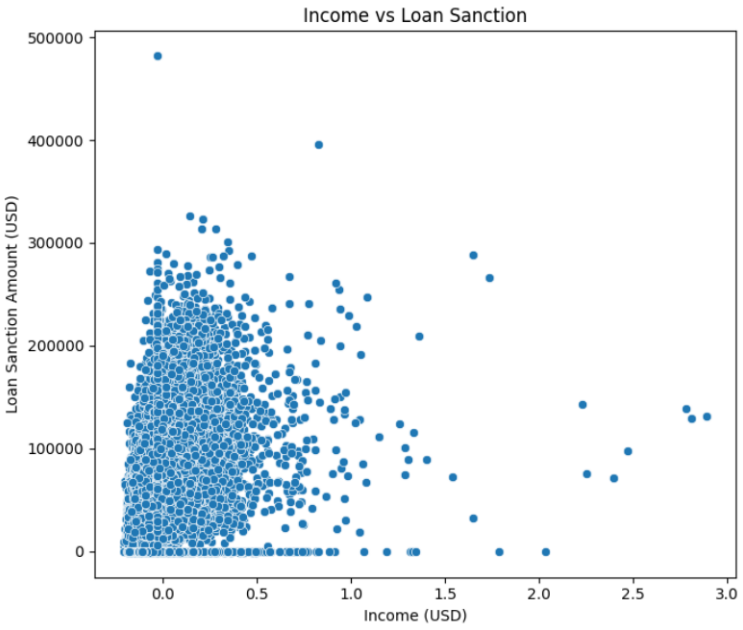

Figure 2: Scatter Plot of Applicant Income vs Loan Amount

Interpretation: There appears to be a weak positive correlation between income and loan amount. Higher income does not always guarantee higher loan amounts, suggesting other features influence the sanctioning process.

7.3 Correlation Heatmap of Features

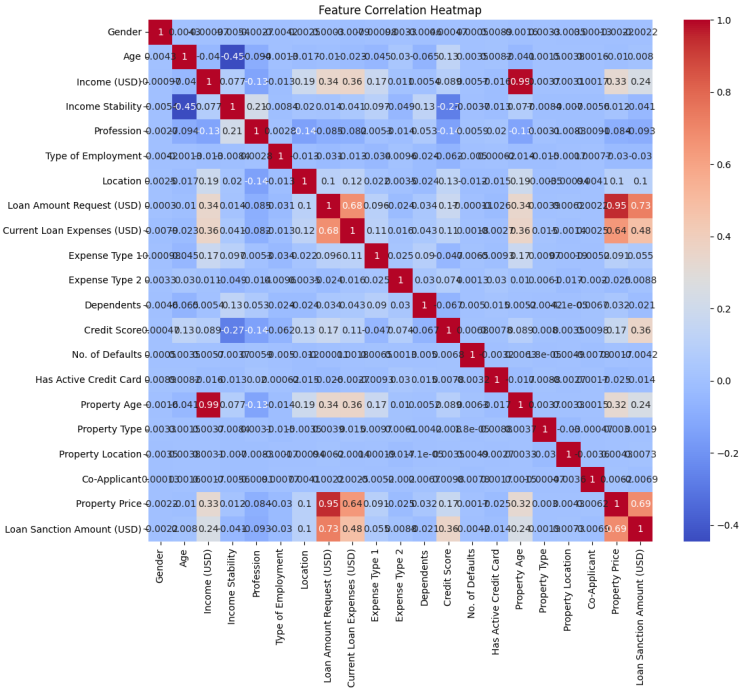

Figure 3: Correlation Heatmap of Dataset Features

Interpretation: The heatmap indicates that loan amount has moderate correlation with some of the features like requested loan amount, income etc.

7.4 Actual vs Predicted Loan Amount

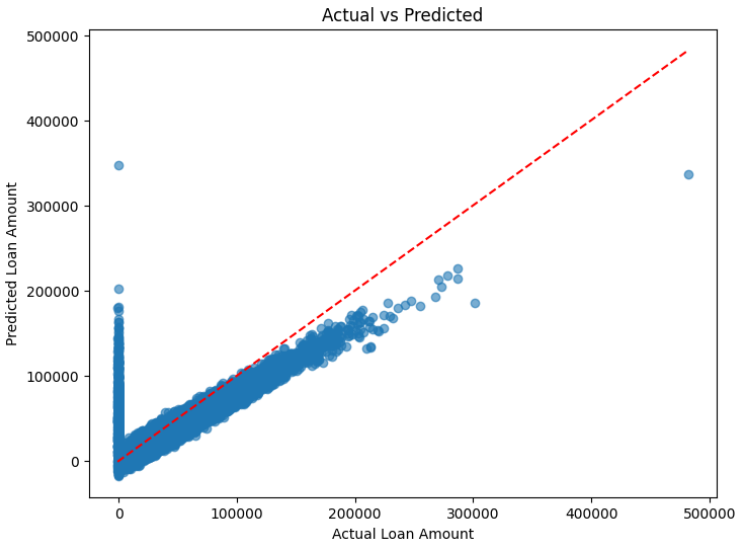

Figure 4: Actual vs Predicted Loan Amount

Interpretation: The points lie close to the diagonal, indicating good prediction performance, except for those having output value of $0.

7.5 Residual Plot

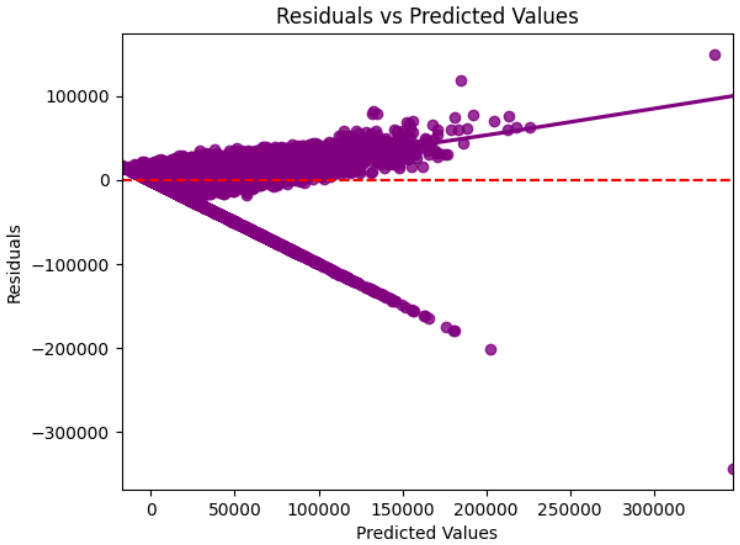

Figure 5: Residuals vs Predicted Values

Interpretation: The residuals appear to form a 'V' pattern, which shows the spread increases as predicted income increases, which implies heteroscedasticity (i.e. variance is not constant).

7.6 Boxplot of Income

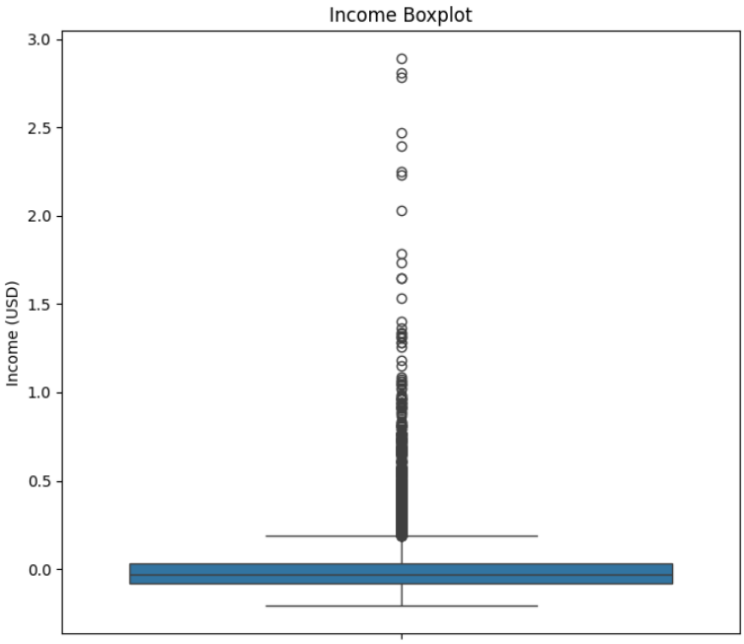

Figure 6: Boxplot of Applicant Income

Interpretation: The plot show the presence of outliers. It has a tighter interquartile range, suggesting most income fall within a narrow band.

7.7 Bar Plot of Feature Coefficients

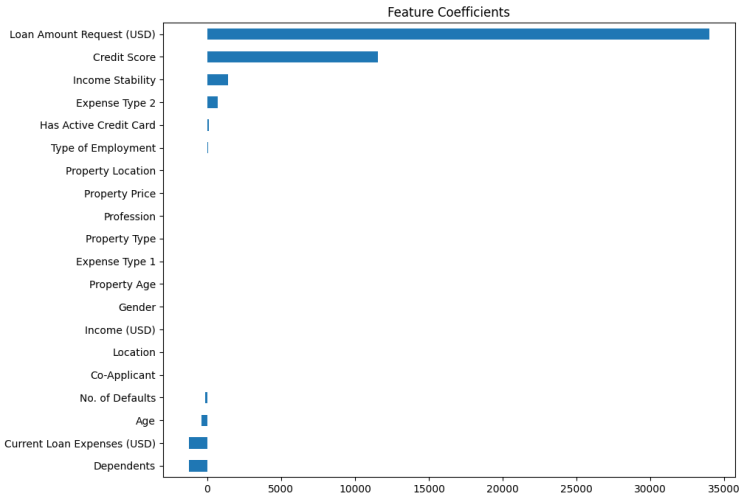

Figure 7: Bar Plot of Feature Coefficients

Interpretation: This plot reveals the influence of each feature in the linear regression model. requested loan amount, credit score, property price seem to contribute the most to the loan amount

prediction.

8 Results Tables

Table 1: Cross-Validation Results (K = 5)

Fold	MAE	MSE	RMSE	R^2 Score	Adjusted R^2 Score
Fold 1	21491.211507	9.641560×10^8	31050.860871	0.571348	0.569897
Fold 2	21284.407862	9.269316×10^8	30445.550920	0.599556	0.598201
Fold 3	21508.741877	9.625275×10^8	31024.627812	0.582933	0.581521
Fold 4	21802.991328	9.775826×10^8	31266.317313	0.596975	0.595611
Fold 5	21623.316627	9.842496×10^8	31372.751910	0.577007	0.575576
Average	21542.133840	9.630894×10^8	31032.021765	0.585564	0.584161

Table 2: Summary of Results

Description	Student's Result
Dataset Size (after preprocessing)	29660
Train/Test Split Ratio	80:20 (K = 5 in K-Fold)
Features Used	'Gender', 'Age', 'Income (USD)', 'Income Stability', 'Profession', 'Type of Employment', 'Location', 'Loan Amount Request (USD)', 'Current Loan Expenses (USD)', 'Expense Type 1', 'Expense Type 2', 'Dependents', 'Credit Score', 'No. of Defaults', 'Has Active Credit Card', 'Property Age', 'Property Type', 'Property Location', 'Co-Applicant', 'Property Price', 'Loan Sanction Amount (USD)'
Model Used	Linear Regression
Cross-validation Used?	Yes
Number of Folds	5
MAE (Test Set)	21542.133840
MSE (Test Set)	9.630894×10^8
RMSE (Test Set)	31032.021765
R^2 Score (Test Set)	0.585564
Adjusted R^2 Score	0.584161
Most Influential Feature(s)	Loan Amount Requested (USD), Credit Score, Property Price
Observations from Residual Plot	Residual spread increases as predicted income increases, which implies heteroscedasticity (i.e. variance is not constant).
Overfitting/Underfitting?	Under Fitting
Justification (if any)	Low performance metrics. Similar performance for both train and test.

9 Best Practices

- Always handle missing values, check data quality and remove any outliers.

- Normalize or scale features before training.
- Use residual plots to check assumptions of linearity and homoscedasticity.
- Evaluate the model using multiple metrics.
- Use cross-validation for better generalization estimates.

10 Learning Outcomes

- Understood the full pipeline of building a regression model.
- Practiced data cleaning, preprocessing, and feature encoding.
- Visualized data and regression results effectively.
- Learned to interpret and explain model performance using standard metrics.
- Compared predicted vs actual outcomes and derived insights.